

Ejercicio 2: Everything as Code y Gestión de Entornos Contexto: Su empresa sufre del síndrome funciona en mi máquina. Los desarrolladores configuran sus entornos locales manualmente, y el equipo de operaciones instala las dependencias en los servidores de forma manual mediante scripts de Bash no versionados.

¿Como el Dockerfile resuelve el problema de la deriva de configuración?

La deriva de configuración ocurre cuando los entornos de desarrollo, pruebas y producción comienzan a diferir silenciosamente entre sí con el paso del tiempo. En los esquemas tradicionales esto sucede porque la infraestructura es mutable; es decir, los servidores se van modificando directamente sobre la marcha mediante parches manuales, instalaciones improvisadas o scripts de Bash que no están bajo control de versiones. Como consecuencia directa, un código que funciona perfectamente en la computadora de un desarrollador termina fallando al llegar al servidor de producción debido a pequeñas discrepancias en las versiones de las librerías, dependencias o configuraciones del sistema operativo.

El Dockerfile resuelve este problema aplicando el principio de infraestructura inmutable bajo la filosofía de Everything as Code (Todo como Código). En lugar de depender de configuraciones manuales u opacas en cada máquina, el Dockerfile funciona como un contrato declarativo versionado en Git que especifica la versión exacta del entorno de ejecución de Node.js, las librerías necesarias, el directorio de trabajo y el puerto de escucha. A partir de este archivo de texto se construye una imagen de Docker que encapsula toda la aplicación y sus dependencias en un artefacto inmutable; esto implica que el entorno no se modifica in situ una vez desplegado, sino que ante cualquier ajuste o actualización de código se genera una nueva imagen estandarizada y se reemplaza la anterior.

De esta manera, se alcanza una paridad estricta entre todos los entornos y la idempotencia en la gestión de infraestructura. La misma imagen exacta que el desarrollador ejecuta localmente en su máquina es la que evalúan los pipelines de integración continua y la que se despliega en los servidores de producción. Al ser un proceso completamente idempotente, construir el contenedor a partir del Dockerfile siempre produce exactamente el mismo resultado operativo en cualquier servidor anfitrión, erradicando por completo el clásico dilema "funciona en mi máquina" y garantizando la reproductibilidad del software.

¿Qué ventajas de trazabilidad y auditoría (conceptos clásicos de SCM) se obtienen al tratar la infraestructura como código en Git, en comparación con los logs manuales de un administrador de sistemas?

Tratar la infraestructura como código en Git reemplaza los volátiles y opacos historiales de consola del administrador de sistemas por un registro inmutable. Cada modificación en los servidores queda asentada mediante un commit que registra con precisión quién realizó el cambio, en qué fecha, qué líneas específicas cambiaron y bajo qué justificación técnica o ticket. Esto resuelve la falta de transparencia de los logs manuales, proporcionando un historial inalterable que simplifica cualquier proceso de auditoría.

Asimismo, la infraestructura como código integra el control de cambios directamente en el flujo de trabajo. A diferencia de las modificaciones improvisadas en el servidor, todo ajuste requiere un Pull Request que exige revisión por pares (Peer Review) y activa un pipeline de Integración Continua (CI) con Quality Gates automatizados. Esta dinámica sustituye las actas manuales tradicionales y le garantiza a los auditores que ningún cambio llegó a producción sin haber sido revisado y probado de manera preventiva.

Por último, el repositorio en Git actúa como la única fuente de la verdad para la infraestructura, erradicando la deriva de configuración (configuration drift). Si ocurre una falla en producción, la auditoría y la recuperación son inmediatas: en lugar de rastrear qué comandos se ejecutaron a mano en la consola, basta con aplicar un git revert para restaurar el entorno completo a un estado estable previo de forma determinista.